

# Lab 1: About Memory and Some Useful Tools

Master M1 MOSIG, Univ. Grenoble Alpes (Ensimag / UFR IM2AG)

2026

The files for this lab are provided on Moodle (`os_lab1.tar.gz`)<sup>1</sup>.

## I. Stack and Heap

The *stack* and the *heap* are two distinct memory segments defined by the system for each process so that they can store their data.

- The stack is used to *automatically* allocate memory for the variables defined within functions. Their size is known at compile time and for such variables, memory is automatically reserved when the program enters the function and released when the program leaves the function. It is thus used for temporary storage of information and the use of this information is restricted to the lifespan of the function call.
- The heap is used to store data whose size is unknown at compile time and/or whose lifetime may be arbitrary (and hence possibly unknown at compile time). The memory for this data is to be managed explicitly by the developer, using standard C functions such as `malloc` and `free` (see `man malloc`).

Consider the following code (provided in `ex1.c`) and answer the question:

```
#include <stdio.h>

int min(int a, int b, int c){
    int tmp_min;
    tmp_min = a <= b ? a : b;
    tmp_min = tmp_min <= c ? tmp_min : c;
    return tmp_min;
}

int main(){
    int min_val = min(3, 7, 5);
    printf("The min is: %d\n", min_val);
    exit(0);
}
```

---

<sup>1</sup>The `tar` command (see `man tar`) can be used to extract the archive. More specifically, use the following command to extract a `tar.gz` file: `tar zxvf myfile.tar.gz`

**Question I.1:** *In which memory segment are the variables `a`, `b` and `c` allocated ? When is the memory allocated to them released ? What about the `tmp_min` variable ?*

Let us now consider the following code (provided in `ex2.c`):

```
int *vect_sum(int *v1, int *v2, int size){
    int *r, i;
    r = malloc(sizeof(int) * size);
    for(i = 0; i < size; i++){
        r[i] = v1[i] + v2[i];
    }
    return r;
}

int main(){
    int v1[] = {1, 2, 4, 7};
    int v2[] = {3, 4, 9, 2};

    int *p_result = vect_sum(v1, v2, 4);
    /* prints the content of the given vector */
    print_vect(p_result, size);
    exit(0);
}
```

**Question I.2:** *What value is contained by variable `r` after the call to `malloc()` inside function `vect_sum()` ? In which memory segment is this value stored ?*  
*By the way, note that such an assignment, i.e., storing the result returned by `malloc` into a pointer variable with a type other than `void*` (here, `int*`) does not require a cast. Indeed, this case, the compiler automatically promotes the result to the appropriate pointer type.<sup>2</sup>*

**Question I.3:** *What is the exact meaning of the assignment: `r[i] = v1[i] + v2[i];` ? This assignment results in a (memory) write instruction. In this program, in which memory segment does this write happen ?*

**Question I.4:** *Write a program that behaves in the same way without using `malloc()`. You may need to change the parameters of function `vect_sum()`.*

**Question I.5:** *What is the life cycle of a stack-allocated variable ? of a heap-allocated variable ?*

## II. Illegal memory accesses

Correct memory allocation is required for each variable to lie at a distinct place in the memory space. Pointers are very useful, but they also allow programmers to attempt accessing memory addresses that have not been allocated. If a program tries to read or write at such an address, it

---

<sup>2</sup>For more information about this topic, you can read the following (optional) links:

- <https://stackoverflow.com/questions/605845/should-i-cast-the-result-of-malloc-in-c>
- <https://gustedt.wordpress.com/2014/04/02/dont-use-casts-i/>

may be killed *by the operating system* with the SIGSEGV signal<sup>3</sup>. A memory access that may raise such a signal is called an *illegal memory access*.

**Question II.1:** Which lines of this piece of code are illegal memory accesses? Which one would raise a warning using a “picky” compiler<sup>4</sup>?

```
1. int *pa = 2;
2. *pa = 34;
3. int b = 4, *pb = &b;
4. *pb = 5;
5. int *pc;
6. printf("pc is equal to %d\n", pc);
7. printf("*pc is equal to %d\n", *pc);
8. pc = malloc(sizeof(int));
9. *pc = -2;
10. pa = pc;
11. free(pa);
12. pc = -4;
```

### III. Pointer arithmetic in C

In this exercise, we consider the program defined by the source file `pointer_arithmetic.c`. Answer the following questions after having read the source code and executed the program.

**Question III.1:** Based on the addresses displayed by the program, compute the distance in bytes between the initial address and the final address pointed to by `p1`. Explain the reason for the result you obtain.

**Question III.2:** Modify `pointer_arithmetic.c` to compute and display the difference between the initial and the final value of the pointer `p1`. You should obtain the same result as the one you computed in the previous question.

**Question III.3:** Observe the difference between the initial and the final value of `p2`. Is the result consistent with the value observed for `p1`?

**Question III.4:** Observe the difference between the initial and the final value of `p3`, `p4`, and `p5`. What can you conclude?

**Question III.5:** As you may have noticed, you are provided with a *Makefile* to automatize the compilation of the programs (More about *Makefiles* to come in the next exercise). To compile the program for this exercise, you can simply run:

---

<sup>3</sup>“SIG” stands for “signal” and “SEGV” stands for “segmentation violation”, a historical expression.

<sup>4</sup>In practice, the *pickiness* of a compiler in raising warnings can be configured through options to activate using flags. A detailed documentation can be found here (<https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html>) for gcc. The provided *Makefile* sets the following flags, which enable a large set of diagnostic messages: `-Wall -Wextra -pedantic`.

```
make pointer_arithmetic.run
```

*You should notice a warning message generated by the compiler. This warning message is generated thanks to the use of the `-Wpointer-arith` compilation flag. Explain the reason for this message.*

**Question III.6:** *Observe the value of `p6` and `p7` as well as the result of the computation `D`. Explain this result.*

## IV. About Makefiles

A Makefile is provided to you in the archive `os_lab1.tar.gz`. Makefiles are used to automatize the compilation of your code using the `make` utility.

In case you don't know about the notion of Makefile, the provided file includes detailed comments that explain how it works. Do not hesitate to look for additional resources on Internet.<sup>5</sup>

To check that you understand how a Makefile works, open the provided Makefile and answer the following questions:

**Question IV.1:** *List the commands that are going to be executed by `make` when the following command is executed:*

```
make ex1.run
```

**Question IV.2:** *Same question for the following command:*

```
make ex2.run
```

*Start by explaining the variables `'$@'` and `'$<'`.*

**Question IV.3:** *Same question for the following command:*

```
make prog_0.run
```

*Start by explaining the use of the symbol `'%'` in a rule.*

**Question IV.4:** *Same question for the following command:*

```
make all
```

**Question IV.5:** *At the end of the Makefile, a phony target (named `clean`) is defined using the keyword `.PHONY`. Explain the purpose of such phony targets.*

---

<sup>5</sup>For example:

- <https://makefiletutorial.com/>
- <https://www.gnu.org/software/make/manual/make.html>

## V. Gdb

gdb is a debugging tool. It allows step-by-step execution during which the user can explore the state of the memory (variables, pointers, registers, stack, ...).

**Question V.1:** *During your training week you (may) have been provided with a simple gdb tutorial. If you have not been through it yet, it is time to do so. This tutorial is provided to you in the file `gdb-tutorial_EN.c`. Open the file and follow the instructions.*

## VI. Valgrind

valgrind is a tool used to track runtime errors. It simulates the execution of a given executable inside a virtual system, and records any illegal access to the memory as well as other errors.

To use it, simply run:

```
$ valgrind ./my_executable
```

The programs (with a "prog\_" prefix) given in the archive for this lab are all syntactically correct C programs, but they all misbehave at run time.

**Question VI.1:** *Use valgrind to find and solve errors in the given C files<sup>6</sup>. valgrind is usually very verbose; write down the valgrind errors that helped you and explain their meaning. (If programs are compiled with the `-g` option valgrind is able to provide the source file name and line where the error happened). You may also find the following options useful:*

- *To use gdb and valgrind together, see <http://valgrind.org/docs/manual/manual-core-adv.html#manual-core-adv.gdbserver-simple>*
- *`--leak-check=yes` (to get information about memory zones that were never freed and are definitely lost);*
- *`--show-reachable=yes` (to get information about memory that were never freed and are still reachable).*

## VII. AddressSanitizer (ASan)

AddressSanitizer is another tool that can detect illegal memory accesses. However, it works differently from valgrind. AddressSanitizer instruments the application source code, and therefore, it requires recompiling the source code.

AddressSanitizer is integrated into gcc since version 4.8. It is for instance actively used in the development of the chromium and firefox web browsers.

To use AddressSanitizer, compile your code with the appropriate flags:

```
$ gcc -g -fsanitize=address my_file.c -o my_exec_file
```

---

<sup>6</sup>You might want to keep a copy of the initial code of the programs to be able to use them in the next exercise.

Then, you can run your program as usual.

**Question VII.1:** *Observe the errors in programs with a "prog\_" prefix using the AddressSanitizer. Obviously, you should use the initial version of the codes, that is, without the bug fixes.*

**Comparing Valgrind and AddressSanitizer:** Both tools can be useful. Each of them may detect errors that the other one is unable to detect. Regarding performance, AddressSanitizer is much more efficient than Valgrind at the cost of requiring to recompile the application.

**Question VII.2:** *If you are interested in learning more about AddressSanitizer, you can have a look at:*

- *The Github repository:* <https://github.com/google/sanitizers/wiki/AddressSanitizer>
- *The main publication related to this work:* <https://research.google.com/pubs/pub37752.html>

## VIII. Observing processes virtual address space layout

**About the proc file system** In a Linux system, the `/proc` directory contains a hierarchy of special files which represent the current state of the kernel.

Among the information that can be found in `/proc`, information about the current state of a process `PID` can be found in the `/proc/PID` directory. For a detailed description of all the information that can be found in `/proc`, we refer you to the man-pages: `man 5 proc`.

Among the virtual files available in `/proc`, we will focus on the information provided by `/proc/PID/maps`. This file describes the state of the virtual address space of a process.

The command `cat` can be used to display the content of a file. Running "`cat /proc/self/maps`" displays information about the virtual address space of the current process.

**Question VIII.1:** *Which process does the displayed information refer to in this case?*

**Question VIII.2:** *We can see that each line of the `maps` file includes several fields. What does each of these fields correspond to?*

**Question VIII.3:** *Observe the position of the stack and the heap in the virtual address space. Based on what you know about their internal details, explain why they are positioned this way.*

Let us now compare the virtual address space of two processes.

**Question VIII.4:** *What differences do you notice between two processes that are run with the exact same command line (same program and same parameters)?*

**Question VIII.5:** *There is a good reason for these differences. Explain.*

**Bonus question 1:** *We can observe that the executable file executed by a process is mapped several times at the beginning of the virtual address space. What do these different mappings correspond to (remember "Lecture 1: Introduction")?*

**Bonus question 2:** *What differences do you observe between these mappings? Does this match with the answer to the previous question? Try to explain.*

We are now going to think about the kernel memory space.

**Bonus question 3:** *Based on your observation of the address space of a process, figure out where is the virtual address space of the kernel.*

**Bonus question 4:** *Why does it belong to the virtual address space of the user although the user does not get access to it?*

## – Bonus –

*Below this point, the exercises are optional.*

## IX. Recursive functions

```
int power(int a, int n){
    if( n != 0 )
        return a*power(a, n - 1);
    else
        return 1;
}
```

**Question IX.1:** *Make a rough estimation of the memory needed to compute `power(2, 3)`.*

**Question IX.2:** *We provide you with the program `rec.c`: try to validate your estimation.*

*To do so, you can use the built-in<sup>7</sup> gcc function `void * __builtin_frame_address(unsigned int level)` that returns the address of the function frame (with `level = 0`, returns the frame address of the current function)<sup>8</sup>*

## X. More of Gdb

**Question X.1:** *We consider again the programs presented in Exercise I. Use `gdb` to visualize the state and the evolution of the heap and the stack of the executed program.*

*Here are some useful `gdb` commands:*

---

<sup>7</sup>Built-in functions are functions not defined in the C standard.

<sup>8</sup>See <https://gcc.gnu.org/onlinedocs/gcc/Return-Address.html>

- `x/nfu addr`  
*x* command is used to display the memory, starting from *addr*; *n*, *f*, and *u* are all optional parameters that specify how much memory to display and how to format it.
  - **n**: A decimal integer (default 1) that specifies how much memory (counting by units *u*) to display.
  - **f**: The display format: 's' (null-terminated string), or 'i' (machine instruction). The default is 'x' (hexadecimal) initially.
  - **u**: the unit size. The unit size is any of 'b' (Bytes), 'h' (Halfwords – two bytes), 'w' (Words – four bytes – default), 'g' (Giant words – eight bytes).
- `info frame`  
 to get information about stack frames.
- `p $sp`  
 to get the value of the stack pointer.
- `x/5i $pc-6`  
 to print 5 instructions 6 words before the current program counter.

**Question X.2:** Try to use `gdb` to better understand memory usage for the recursive program introduced in Exercise VIII.

## XI. Observing the calls made by a program

Tools for observing the interactions of an application with the operating system (system calls and libraries) can be useful. The tools `strace` and `ltrace`<sup>9</sup> are two of them. Look at their documentation and observe the obtained output when run with one of the program manipulated during this lab (for instance, `ex1.run`).

### – Additional Tips –

Please find below a few additional tips, which can be useful for this lab and the next ones.

## XII. Automatically tracking dependencies to header files using GCC and Makefiles

In practice, it may happen that a `.c` source code file depends on several `.h` files. In this case, a complete Makefile should know about these dependencies to ensure that the `.c` file is recompiled if one of the corresponding `.h` files is modified.

Instead of manually keeping track of these dependencies, it is possible to offload this burden to the C compiler and the Makefile by using simple changes in the Makefile. See for instance the following documentation: <https://nathandumont.com/blog/automatically-detect-changes-in-header-files-in-a>

---

<sup>9</sup>Note that on some recent Linux distributions, `ltrace` is not available or does not work correctly. Other tools can be used instead, such as `latrace` or `uftrace`. `uftrace` requires programs to be compiled with the `-pg` option, which provides the necessary support for profiling.



### XIII. Linters

Linters are tools for detecting problematic code patterns (i.e., pieces of code that are buggy or dirty or useless or complex or error-prone). OCLint is an example of such a tool for C programs. Check it out!

### XIV. Understanding function signatures in man pages

Man pages are very useful to obtain documentation about system calls and library functions, among other things.

However, the C syntax used in recent versions of Linux man pages (such as the ones visible on this site and probably also the ones installed on your own Linux machine) can be sometimes confusing. The goal of this section is to help you understand this syntax. To use a concrete example, we will study the `mmap()` function, which will be useful in several labs during the semester (for now, it is not necessary to know what this function does nor how to use it).

If you look at a recent version on the Linux man page for the `mmap()` system call (man 2 `mmap`), you will see the following synopsis:

```
void *mmap(size_t length;
           void addr[length], size_t length, int prot, int flags, int fd, off_t offset);
```

The syntax of this synopsis must be understood as follows:

- On the first line, we begin (as usual) with the return type and the name of the function.
- Then, still on the first line, we have a list with zero, one or several parameters, ending with a semicolon. These elements (if any) correspond to forward declarations of parameters. Here, we can notice that we have one forward declaration for a parameter named `length`.
- After the forward declarations (if any), we have the “real”, ordered list of parameter declarations. Here, this corresponds to the contents of the second line, and we can notice that we have a total of 6 parameters, separated with comas (instead of semicolons).
- A parameter surrounded by square brackets corresponds to the size of an array.
- Therefore, we can conclude that `mmap()` has six parameters, in the following order and with the following types:

- `void *addr`: pointer to an array of bytes<sup>10</sup>
- `size_t length`: the number of elements in the array
- `int prot`
- `int flags`
- `int fd`

---

<sup>10</sup>Note that, given that we have a generic pointer (`void*`), the signature in itself is not sufficient to deduce that we are manipulating an array of *bytes*. Reading the textual description of the man page is necessary for that matter.

– `off_t offset`

As additional examples, you can also take a look at the man page of the `read()` system call or the `memcpy()` library function.<sup>11</sup>

For more information on this matter, see for instance the following document from the C language specification working group: “WG14 N3433 — Alternative syntax for forward declaration of parameters”.

---

<sup>11</sup>Note that the syntax used for the synopsis of these functions can be misleading and is just a notation, as it is not compliant with the C syntax. Indeed, declaring an array of `void` is forbidden. These functions actually manipulate arrays of *bytes*, as explained in the textual description within their man pages.